Task 2: Matrix Multiplication for Inference

CS683 Programming Assignment 1

Dhurandhar Programming

Advanced Computer Architecture
Autumn 2026

7 September 2026

Contents

1 Overview **2**
1.1 Graded result (official harness, $n = 1024$) . . . 2

2 Experimental setup **2**
2.1 Hardware and software . . . 2
2.2 Measurement method . . . 2
2.3 Kernels used in the experiments . . . 3

3 Kernel design **3**
3.1 The register tile . . . 3
3.2 The three submitted stages . . . 4

4 Task 2A: Software prefetching **4**
4.1 2A.1 Impact of the matrix size . . . 4
4.2 2A.2 CPU metrics . . . 5
4.3 2A.3 Execution time and speedup . . . 6
4.4 2A.4 Optimal parameters . . . 7
4.5 2A.5 Effect of the hardware prefetchers . . . 7
4.6 Questions . . . 8

5 Task 2B: SIMD **8**
5.1 2B.1 Impact of the SIMD instruction width . . . 8
5.2 2B.2 Impact of the matrix size . . . 9
5.3 2B.3 Instruction count . . . 9
5.4 2B.4 Execution time and speedup (Table 2.1) . . . 10
5.5 Questions . . . 10

6 Task 2C: Software prefetching + SIMD **10**
6.1 2C.1 The Task 2A analyses with SIMD . . . 10
6.2 2C.2 The Task 2B analyses with prefetching . . . 15
6.3 Questions (both techniques applied) . . . 15
6.4 Final performance summary . . . 16

7 README deliverables **16**

8 Conclusions **17**

Overview

This report follows the structure of the problem statement. Task 2A tunes software prefetching (prefetch distance and cache fill level) on the scalar SGEMM kernel, measures cache misses and software-prefetch counts with `perf`, and studies the hardware prefetchers by switching them off. Task 2B compares SIMD widths, their instruction counts and speedups (Table 2.1). Task 2C repeats both analyses with prefetching and SIMD applied together and ends with the final performance summary plot. Every numbered item of the statement has a subsection of the same name and every question is answered in a shaded box.

The workload is single-precision matrix multiplication in the "NT" layout used by `ggml/llama.cpp`: $C[i][j] = \sum_p A[i][p]\,B[j][p]$, with A of size $M \times K$, B of size $N \times K$ and the contraction dimension K contiguous in both operands. The kernels accept arbitrary M, N, K and leading dimensions, which is required because `llama.cpp` calls them with its own tensor shapes and strides. The performance sweeps use square problems $M = N = K = n$ so that a single number n describes the working set on each plot axis.

Graded result (official harness, $n = 1024$)

Stage	Median time (ms)	GFLOP/s	Speedup
`matmul_naive` (reference)	2818.3	0.76	1.00×
`matmul_simd` (AVX2, 4×3 register tile)	78.2	27.5	36.0×
`matmul_prefetch` (tiling + row-head prefetch)	63.9	33.6	44.1×
`matmul_optimized` (tiling + K-blocking + prefetch)	64.6	33.2	43.6×

All three stages are correct (30/30) and the optimized kernel scores 100/100.

Experimental setup

Hardware and software

Item	Configuration
Processor	Intel Core i5-8265U, 4 cores / 8 threads; reported maximum clock 1.6 GHz on this laptop
L1 data cache	32 KiB per core, 8-way, 64 B lines
L2 cache	256 KiB per core, 4-way
L3 cache	6 MiB shared, 12-way
SIMD support	SSE4.2, AVX2 + FMA; no AVX-512
Hardware prefetchers	L2 streamer, L2 adjacent-line, L1 next-line, L1 IP-stride (MSR `0x1A4`)
Operating system	Ubuntu 22.04.5 LTS, Linux 6.8.0
Compiler	GCC 11.4.0
Submitted flags	`-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`
Profiling tool	`perf` 6.8.12

Measurement method

The assignment Makefile and flags were not modified. Every timing process was pinned to logical CPU 2 with `taskset -c 2`. Each configuration ran one warm-up call followed by five timed calls and reported the median; the naive kernel, which needs 25 s per call at $n = 2048$, was timed once at that size. Because this laptop's clock moves with temperature, every configuration was measured in three interleaved rounds and the **best median** is reported, which is the estimator least affected by throttling. Speedup is always

$$\text{speedup} = \frac{T_{\text{without the technique}}}{T_{\text{with the technique}}} \quad \text{at the same } (M, N, K),$$

and unless stated otherwise the denominator's reference is the naive scalar kernel. Absolute GFLOP/s are low because of the 1.6 GHz clock; conclusions rest on within-run speedups and counter ratios.

Hardware counters were collected with `perf stat` for `instructions`, `cycles`, `L1-dcache-load-misses`, `l2_rqsts.miss` and `LLC-load-misses`, plus the three software-prefetch events `sw_prefetch_access.t0`, `.t1_t2` and `.nta`. Each kernel was called enough times to dominate the process, a setup-only run (allocation and random fill) was subtracted, and the difference was divided by the number of calls, so every counter in this report is **per SGEMM call**. The hardware prefetchers were switched off by writing `0xF` to MSR `0x1A4` on every core (all four bits set) and restored afterwards.

Kernels used in the experiments

The graded files are `src/matmul_simd.cpp`, `src/matmul_prefetch.cpp` and `src/matmul_optimized.cpp`. Compiled as the assignment Makefile compiles them (no extra flags) they give the submitted kernels. For the analyses, the first two files also accept compile-time flags that switch a single technique on or off, so every experiment variant is the graded source built with `-D` options (`experiments/build.sh` produces one small binary per variant):

Flag(s)	File	Effect
none	both	the submitted kernels: AVX2 4×3 tile; tiling + two-line row-head prefetch with T0
`-DSIMD_WIDTH=128`	both	the same tile and loops with 128-bit SSE (`_mm_mul_ps`+`_mm_add_ps`), built with `-msse4.2`
`-DSCALAR_PREFETCH`	prefetch	Task 2A: the naive scalar loop plus `_mm_prefetch` on both operand rows, one per 64 B line
`-DPREFETCH_DISTANCE=`d	prefetch	in-loop prefetch of all seven operand streams d floats ahead (the distance of the scalar variant too)
`-DPREFETCH_LINES=`ℓ	prefetch	ℓ lines prefetched at the head of the next tile's rows; 0 gives "SIMD + tiling" with no prefetch
`-DPREFETCH_HINT=_MM_HINT_x`	prefetch	the locality hint, T0, T1, T2 or NTA

The report labels the variants as follows: `naive`, `simd`, `prefetch` and `optimized` are the submitted stages; `scalar_pf` is the scalar kernel with prefetching (Task 2A); `sse128` is the 128-bit SIMD stage (Task 2B); `blocked_simd` and `blocked_sse128` are the prefetch stage with non-default distance, line count or hint at 256 and 128 bits (Task 2C).

Kernel design

The register tile

The K loop is a unit-stride dot product of an A row and a B row. The micro-kernel computes a 4×3 block of C at once: per step of eight floats it loads three B vectors and, for each of four A rows, one A vector followed by three FMAs, so **seven loads feed twelve FMAs**. Twelve accumulators, three B vectors and one live A vector use exactly the sixteen AVX2 registers. After the loop each accumulator is reduced with a horizontal sum; a $K \bmod 8$ tail and rows or columns that do not fill a tile use scalar code and a single vectorised dot product. The tile is written out by hand: an array-of-accumulators version under `-O2` kept the array in memory and ran at about 10 GFLOP/s.

Tile $M_R \times N_R$	ymm registers	FMAs per load	Time at 1024^3 (ms)	GFLOP/s
4×3	16	1.71	85	25.2
3×4	17	1.71	105	20.4
6×2	15	1.50	115	18.7
4×2	11	1.33	122	17.6
8×1	10	0.89	130	16.5
4×4	21	2.00	141	15.3
5×3	19	1.88	151	14.2

The register budget dominates: 4×4 and 5×3 have the best arithmetic intensity on paper but spill and finish behind 4×2. More A rows beat more B rows at equal area because only one A vector is live at a time.

The three submitted stages

- **`matmul_simd`** sweeps four A rows against every B row. The four A rows stay in L1, but every B row is re-read for each group of four A rows.
- **`matmul_prefetch`** blocks M and N into panels of at most 96 KiB each, so an A panel and a B panel co-reside in the 256 KiB L2, and prefetches two lines at the head of the next tile's A and B rows with `_MM_HINT_T0`. A new row is a jump to a new page that the hardware next-line prefetcher cannot anticipate; the sequential stream inside a row is left to the hardware.
- **`matmul_optimized`** keeps a B panel of 192 KiB in L2 and sweeps the whole of M through it, so each B panel is loaded from memory once. K is split into blocks of at most 1024 so that the four active A rows (16 KiB) stay inside the 32 KiB L1 at large K; the first K block stores its tile results and later blocks accumulate into C. The single prefetch site is the head of the next A row block.

Task 2A: Software prefetching

Setup. The scalar kernel is instrumented with `_mm_prefetch` on both operand rows, issued once per cache line (every 16 floats) at a distance d ahead of the current index and with one of the four locality hints. Distance was swept over $d \in \{16, 32, 64, 128, 256, 512\}$ floats (1 to 32 lines) and the fill level over `T0`, `T1`, `T2` and `NTA`, at $n \in \{256, 512, 1024, 2048\}$. The `sw_prefetch_access` events confirm that the hints reach the hardware and that the count matches the design ($2 \cdot n^3/16$ prefetches per call):

Hint, $n = 1024$**,** $d = 64$	`sw_prefetch_access.t0`	`.t1_t2`	`.nta`	**expected**
`_MM_HINT_T0`	135.4 M	0	0	134.2 M
`_MM_HINT_T1`	0	135.5 M	0	134.2 M
`_MM_HINT_T2`	0	135.1 M	0	134.2 M
`_MM_HINT_NTA`	0	0	135.2 M	134.2 M

2A.1 Impact of the matrix size

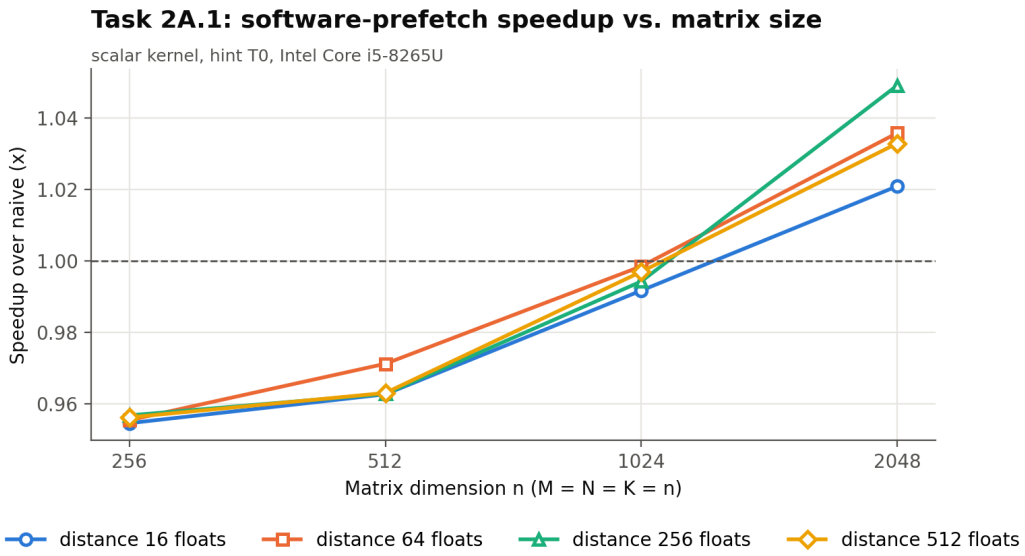


Figure 1: Speedup of the scalar kernel with software prefetching over the scalar kernel without it, versus matrix size, for four prefetch distances (hint T0).

Software prefetching does not help the scalar kernel until the matrices no longer fit the last-level cache. For $n \leq 512$ it is a slight *slowdown* (0.955–$0.971\times$), at $n = 1024$ it is neutral (0.992–$0.998\times$), and only at $n = 2048$, where B alone is

16 MiB against a 6 MiB L3, does it gain 1.02–$1.05\times$. The trend is the same for every distance; the distance only moves the curve by a few percent.

2A.2 CPU metrics

Task 2A.2: cache misses per call, scalar kernel with and without prefetch

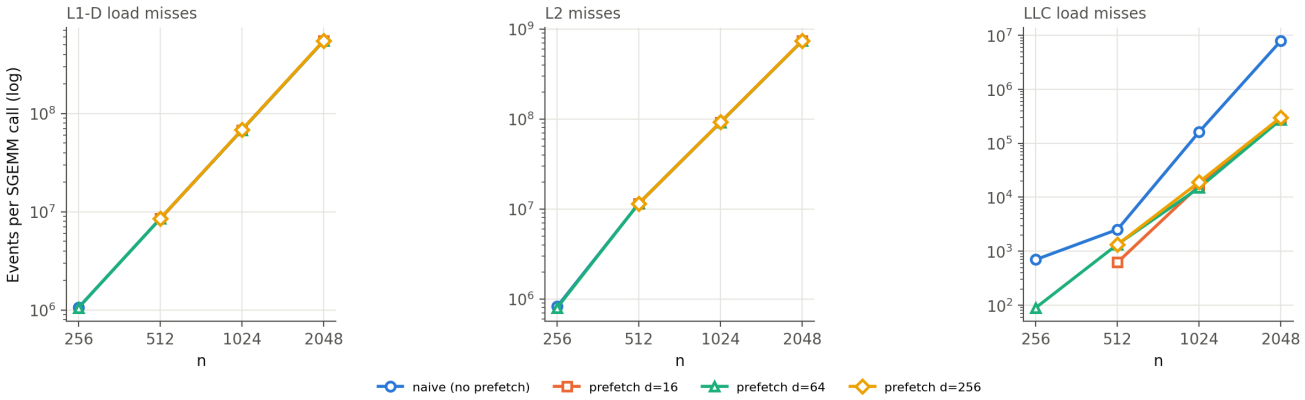


Figure 2: Cache misses per SGEMM call (log scale) for the scalar kernel without prefetching and with prefetching at three distances.

n	Configuration	Instr. (M)	Cycles (M)	L1-D misses (M)	L2 misses (M)	LLC misses (M)	SW prefetch (M)
512	no prefetch	676.8	514.6	8.50	11.52	0.003	0
512	$d = 16$, T0	1230.9	528.0	8.51	11.54	0.001	17.2
512	$d = 64$, T0	1230.9	527.6	8.51	11.55	0.001	17.1
512	$d = 256$, T0	1231.0	526.7	8.51	11.53	0.001	17.1
1024	no prefetch	5403.4	4239.5	68.0	92.0	0.163	0
1024	$d = 16$, T0	9834.6	4278.8	68.0	92.0	0.017	135.7
1024	$d = 64$, T0	9834.6	4271.5	68.0	92.0	0.015	135.4
1024	$d = 256$, T0	9835.2	4303.8	68.3	92.3	0.019	135.3
1024	$d = 64$, T1	9835.1	4275.8	68.0	105.8	0.021	135.5
1024	$d = 64$, T2	9834.7	4274.1	68.0	105.7	0.015	135.1
1024	$d = 64$, NTA	9834.6	4279.5	68.1	261.3	0.035	135.2
2048	no prefetch	43157.0	34863.9	546.6	736.3	7.950	0
2048	$d = 16$, T0	78616.6	35145.9	549.0	740.6	0.277	1078.9
2048	$d = 64$, T0	78592.0	34396.4	546.6	736.6	0.276	1078.5
2048	$d = 256$, T0	78596.1	34400.5	549.7	739.5	0.304	1078.3
2048	$d = 64$, T1	78598.1	34384.5	545.5	844.3	0.270	1078.2
2048	$d = 64$, T2	78594.8	34389.3	545.4	844.8	0.282	1077.7
2048	$d = 64$, NTA	78597.4	34602.8	550.2	2089.0	0.593	1080.1

Three observations. (i) **L1-D and L2 misses are unchanged by prefetching** at every size and distance: misses grow with n^3 because each B row streams past every A row, and a software prefetch does not remove a miss, it only moves it earlier. (ii) **LLC demand misses collapse** (0.163 M to 0.015 M at $n = 1024$, 7.95 M to 0.28 M at $n = 2048$) because the prefetch instruction, not the demand load, now takes the miss; this is the only counter that reflects prefetching and it is why the $n = 2048$ case gains a few percent. (iii) **The fill level changes where the line lands**: T1/T2 raise L2 misses by 15% and NTA by $2.8\times$ (92 M to 261 M at $n = 1024$), because `prefetchnta` brings the line to L1 without filling L2, so it misses L2 again when it is evicted from the small L1. The instruction count rises by 82% (the per-element line test plus the prefetch itself) while the cycle count rises by under 1%.

2A.3 Execution time and speedup

n	Without SW prefetch (ms)	Best with SW prefetch (ms)	Configuration	Speedup
256	40.6	42.5	$d = 256$, T0	0.96×
512	337.1	347.1	$d = 64$, T0	0.97×
1024	2828.6	2831.2	$d = 64$, T0	1.00×
2048	23530.2	22430.3	$d = 256$, T0	1.05×

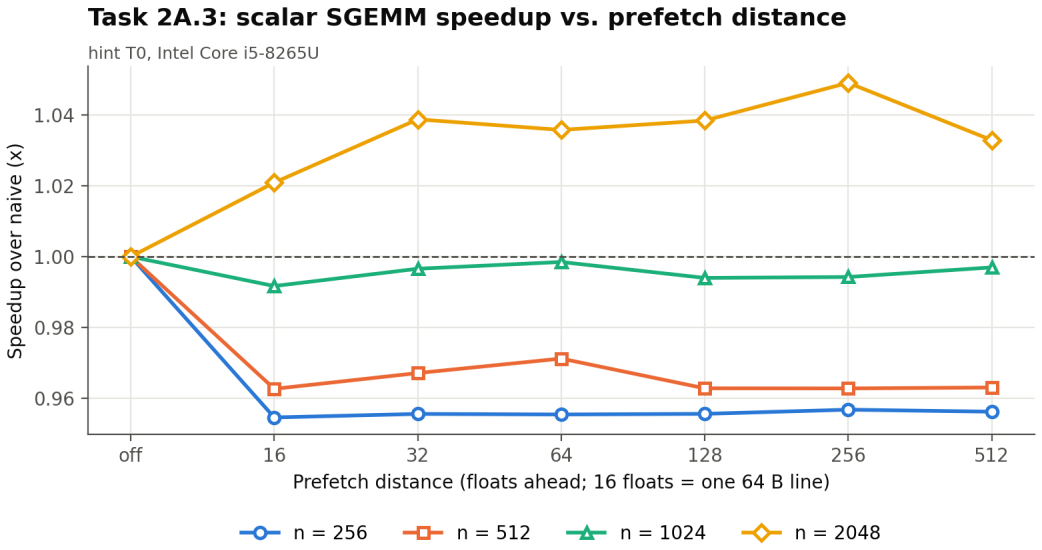


Figure 3: Scalar kernel speedup versus prefetch distance (hint T0). Distance is in floats ahead of the current K index; 16 floats is one cache line.

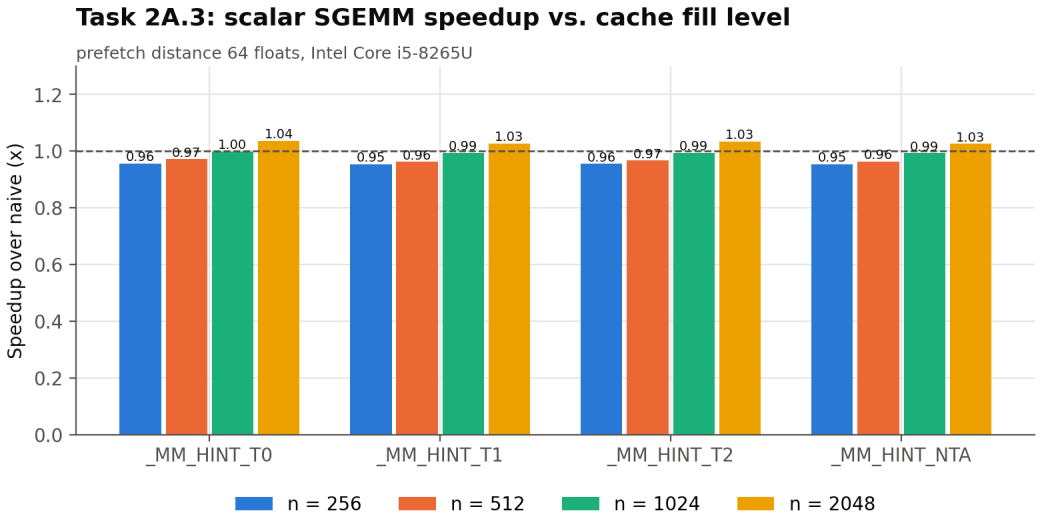


Figure 4: Scalar kernel speedup versus cache fill level at $d = 64$.

Why the scalar kernel cannot profit: its inner loop is `acc += a[p]*b[p]`, a dependent chain with a four-cycle latency per element, so one 64 B line takes $16 \times 4 = 64$ cycles to consume. The hardware L2 streamer keeps up with that pace easily within a page, so the demand loads already hit. The extra prefetch instructions execute in the shadow of the latency chain, which is why 82% more instructions cost under 1% more cycles. At $n = 2048$ the operands exceed the L3

and DRAM latency (≈300 cycles) is no longer fully hidden across page boundaries, so the prefetch that runs 16 lines ahead (256 floats, one page) starts to pay. The difference between hints is inside 1%, in the order T0 > T2 > T1 > NTA at $n = 2048$ (1.036, 1.033, 1.027, $1.026\times$).

2A.4 Optimal parameters

- **Prefetch distance:** 256 floats (16 lines, 1 KiB) at $n = 2048$, giving $1.049\times$; at $n \leq 1024$ no distance beats the kernel without prefetching, and 64 floats is the least harmful.
- **Cache fill level:** `_MM_HINT_T0`. It is marginally the fastest, and the counters show it is the only hint that does not add L2 traffic.

2A.5 Effect of the hardware prefetchers

Task 2A.5: effect of the hardware prefetchers on the scalar kernel

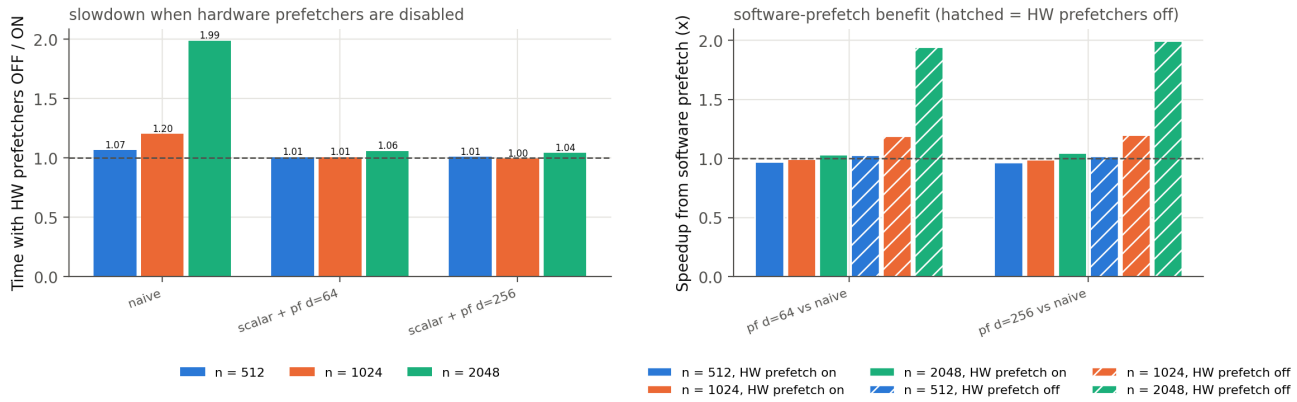


Figure 5: Left: slowdown of each configuration when all four hardware prefetchers are disabled. Right: speedup from software prefetching relative to the naive kernel measured in the same state, with the prefetchers on (solid) and off (hatched).

Configuration	$n = 512$ (ms)		$n = 1024$ (ms)		$n = 2048$ (ms)	
	HW on	HW off	HW on	HW off	HW on	HW off
naive	337	360	2829	3406	23530	46794
scalar + pf d=64	347	350	2831	2854	22718	24062
scalar + pf d=256	349	352	2850	2838	22430	23419
SW prefetch speedup (d=64)	0.97×	1.03×	1.00×	1.19×	1.04×	**1.94**×
SW prefetch speedup (d=256)	0.97×	1.02×	0.99×	1.20×	1.05×	**2.00**×

With the hardware prefetchers off, the naive kernel slows by $1.07\times$ at $n = 512$, $1.20\times$ at $n = 1024$ and $1.99\times$ at $n = 2048$: once B streams from DRAM the per-line demand miss exceeds the 64 cycles of arithmetic per line. The kernel with software prefetching barely changes (1.01–$1.03\times$), because its prefetches now do the job the hardware was doing: LLC demand misses at $n = 1024$ go from 6.62 M (naive, prefetchers off) back to 0.03 M. Software prefetching therefore gives 1.94–$2.00\times$ at $n = 2048$ when the hardware prefetchers are off, against 1.04–$1.05\times$ when they are on. **Software prefetching is a substitute for the hardware prefetchers, not a complement**: on this machine the two cover the same sequential streams, and the hardware version is cheaper because it consumes no issue slots.

Questions

Q1. What trend do you observe in speedup with different matrix sizes?

Speedup rises monotonically with matrix size but stays close to $1\times$: $0.96\times$ at $n = 256$, $0.97\times$ at 512, $1.00\times$ at 1024 and $1.05\times$ at 2048. Below the L3 capacity the prefetch instructions are pure overhead because the scalar loop is bound by its floating-point latency chain and the hardware prefetchers already hide the misses; software prefetching only starts to pay once the operands exceed the 6 MiB L3 and DRAM latency appears. With the hardware prefetchers disabled the same trend is much stronger ($1.03\times$, $1.19\times$, $1.94\times$ at 512, 1024, 2048).

Q2. What is the best prefetch distance?

256 floats, i.e. 16 cache lines or 1 KiB ahead, which gives the maximum $1.049\times$ at $n = 2048$. Shorter distances (16–32 floats) do not cover DRAM latency and longer ones (512 floats) start evicting useful lines; for $n \leq 1024$ the distance does not matter because no prefetching beats none.

Q3. At what cache fill level do you achieve the maximum speedup?

`_MM_HINT_T0`, which fills all cache levels. The differences are within 1% ($1.036\times$ T0, $1.033\times$ T2, $1.027\times$ T1, $1.026\times$ NTA at $n = 2048$), but the counters show why T0 is the right default: T1/T2 raise L2 misses by 15% and NTA by $2.8\times$ because the prefetched line bypasses L2 and must be fetched again after it leaves the 32 KiB L1.

Task 2B: SIMD

Setup. Both SIMD kernels use the same 4×3 register tile; only the register width differs. The 128-bit version uses `_mm_loadu_ps`, `_mm_mul_ps`, `_mm_add_ps` and a `_mm_movehl_ps`/`_mm_shuffle_ps` horizontal sum and is built with `-msse4.2`. The 256-bit version, which is the submitted `matmul_simd`, uses `_mm256_loadu_ps`, `_mm256_fmadd_ps` and an `_mm256_extractf128_ps` + SSE horizontal sum, built with the pinned `-mavx2 -mfma`. Unaligned loads are used because `llama.cpp` passes arbitrary strides; the FMA intrinsic replaces a multiply and a dependent add with one instruction and one rounding. This processor has no AVX-512, so a 512-bit variant could not be measured.

2B.1 Impact of the SIMD instruction width

At $n = 1024$ the 128-bit kernel is $15.0\times$ faster than scalar and the 256-bit kernel $39.5\times$; doubling the register width gives $2.6\times$, more than the $2\times$ of the width alone, because the AVX2 build also uses FMA (one instruction per multiply-add) and its wider registers halve the number of loads per FLOP.

2B.2 Impact of the matrix size

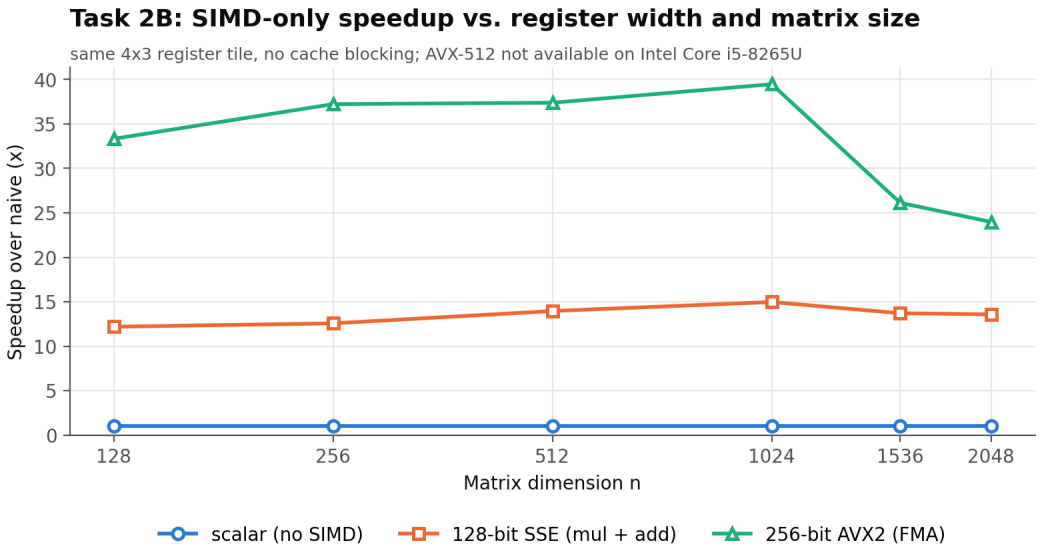


Figure 6: SIMD-only speedup over the scalar kernel for 128-bit and 256-bit registers versus matrix size (no cache blocking).

The 128-bit kernel holds 12–15× at every size. The 256-bit kernel holds 33–40× up to $n = 1024$ and drops to 26× at $n = 1536$ and 24× at $n = 2048$. Once A and B (16 MiB each) exceed the L3, the un-blocked sweep re-reads every B row from DRAM for each group of four A rows; the AVX2 arithmetic is fast enough for that traffic to become the bottleneck, whereas the SSE kernel computes slowly enough to hide it.

2B.3 Instruction count

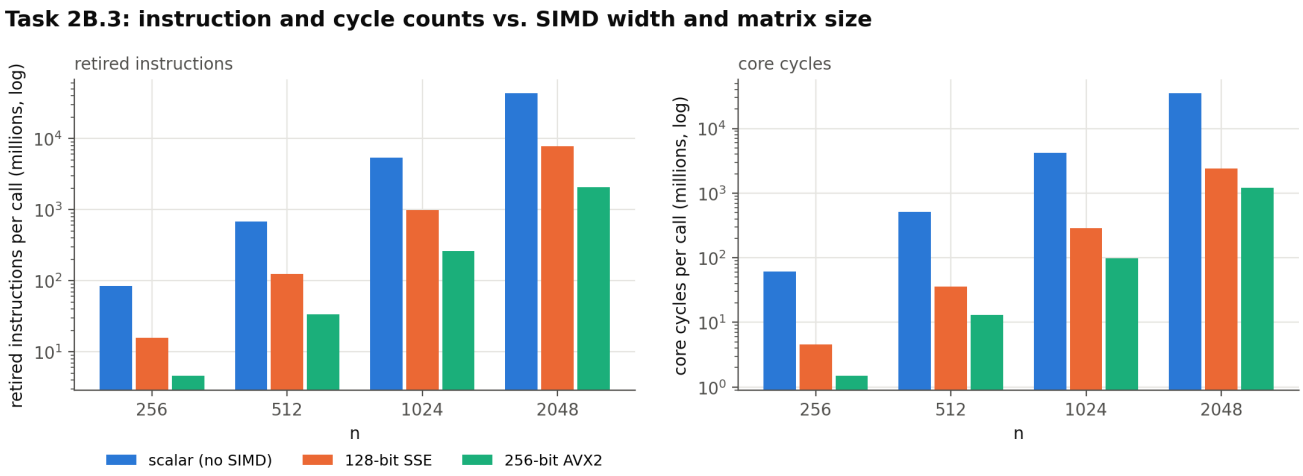


Figure 7: Retired instructions and core cycles per SGEMM call (log scale) for the scalar, 128-bit and 256-bit kernels.

2B.4 Execution time and speedup (Table 2.1)

Metric	SIMD width	Matrix size (M, N, K) $(256, 256, 256)$	$(512, 512, 512)$	$(1024, 1024, 1024)$	$(2048, 2048, 2048)$
No SIMD: instructions (M)	–	84.9	676.8	5403.4	43157.0
No SIMD: execution time (ms)	–	40.63	337.07	2828.57	23530.16
SIMD: instructions (M)	128-bit	15.8	123.7	977.9	7791.1
SIMD: execution time (ms)	128-bit	3.23	24.15	188.96	1732.92
Speedup (normalized to no SIMD)	128-bit	12.6×	14.0×	15.0×	13.6×
SIMD: instructions (M)	256-bit	4.6	33.8	259.2	2055.2
SIMD: execution time (ms)	256-bit	1.09	9.02	71.68	981.56
Speedup (normalized to no SIMD)	256-bit	37.3×	37.4×	39.5×	24.0×
Instruction reduction	128 / 256-bit	5.4× / 18.5×	5.5× / 20.0×	5.5× / 20.8×	5.5× / 21.0×

The instruction reduction exceeds the vector width because the register tile also removes loads: the scalar loop performs two loads per multiply-add, the 4×3 AVX2 tile seven loads per twelve 8-wide FMAs. At $n = 1024$ cycles fall from 4240 M to 284 M (SSE) and 99 M (AVX2), a $43\times$ reduction that exceeds the $20.8\times$ instruction reduction because the scalar loop's IPC (1.27) is limited by its dependent accumulation chain while the tile keeps twelve independent chains in flight (IPC 3.4 for SSE, 2.6 for AVX2). The statement also asks how speedup varies with software-prefetching parameters; the SIMD-only kernels contain no prefetching, and that interaction is measured in Task 2C.

Questions

Q1. What trends do you observe in speedup for different combinations of matrix sizes and SIMD widths?

Speedup grows with register width at every size (12–$15\times$ for 128-bit, 33–$40\times$ for 256-bit). Against size, the 128-bit kernel is flat, while the 256-bit kernel is flat up to $n = 1024$ and then falls to $24\times$ at $n = 2048$; its advantage over 128-bit shrinks from $2.6\times$ to $1.8\times$. The wider kernel computes fast enough to become memory-bound once the operands exceed the L3 (LLC misses per call rise from 0.06 M at $n = 1024$ to 4.7 M at $n = 2048$), so the gain from width is capped by memory bandwidth for large matrices unless cache blocking is added (Task 2C).

Q2. For which SIMD width do you achieve the maximum speedup?

256-bit AVX2, at every size: $39.5\times$ at $n = 1024$ and $24.0\times$ at $n = 2048$, against $15.0\times$ and $13.6\times$ for 128-bit SSE. A 512-bit variant could not be measured on this processor; from the trend it would help only in the cache-resident regime, since at large n the 256-bit kernel is already limited by memory rather than by arithmetic.

Task 2C: Software prefetching + SIMD

Setup. Software prefetching now goes into the vectorised kernel. Because the SIMD kernel is memory-bound at large n (Task 2B), cache tiling is applied as well, and the two prefetch styles are switchable: *in-loop* (the seven operand streams prefetched d floats ahead, once per line) and *row-head* (ℓ lines at the start of the next tile's rows). The sweeps of Task 2A and the width comparison of Task 2B are repeated on this kernel.

2C.1 The Task 2A analyses with SIMD

Impact of the matrix size

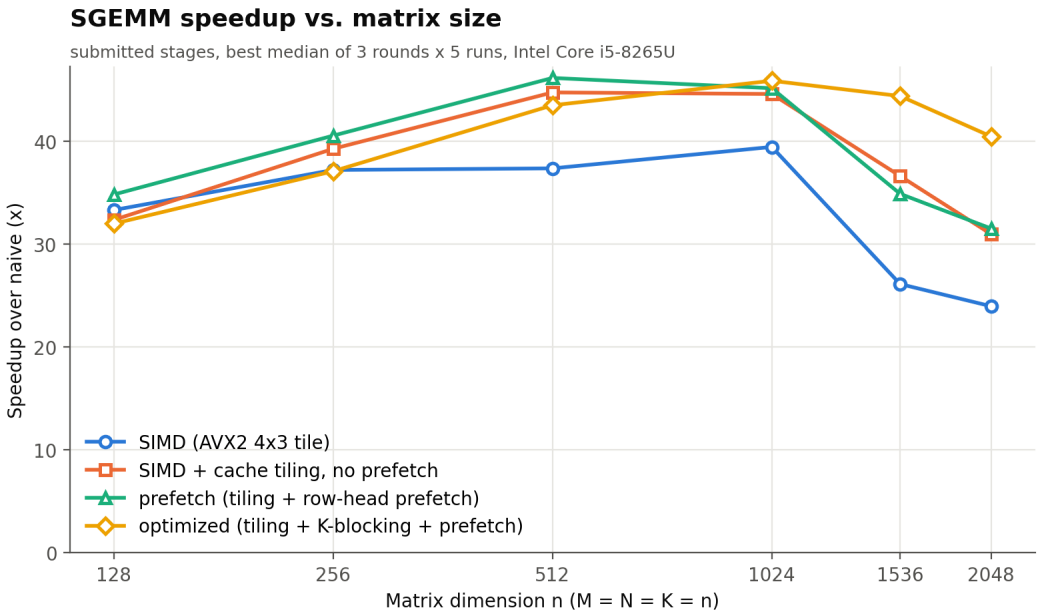


Figure 8: Speedup of the submitted stages over the scalar kernel versus matrix size (README plot 1).

Speedup over naive	n=128	256	512	1024	1536	2048
SIMD (AVX2)	33.3×	37.2×	37.4×	39.5×	26.1×	24.0×
SIMD + tiling, no prefetch	32.4×	39.3×	44.8×	44.6×	36.6×	31.0×
SIMD + tiling + row-head prefetch (`matmul_prefetch`)	34.8×	40.6×	46.2×	45.2×	34.9×	31.5×
SIMD + tiling + K-blocking + prefetch (`matmul_optimized`)	32.0×	37.1×	43.5×	45.9×	44.4×	40.5×

With SIMD the picture of Task 2A inverts: it is *tiling* that tracks the matrix size. Blocking lifts $n = 1024$ from $39.5\times$ to $44.6\times$ and $n = 2048$ from $24.0\times$ to $31.0\times$; the sparse row-head prefetch adds one or two percent on top; and K-blocking, which keeps a 4×1024 block of A (16 KiB) in the 32 KiB L1, is what holds 40–$44\times$ beyond $n = 1536$.

CPU metrics

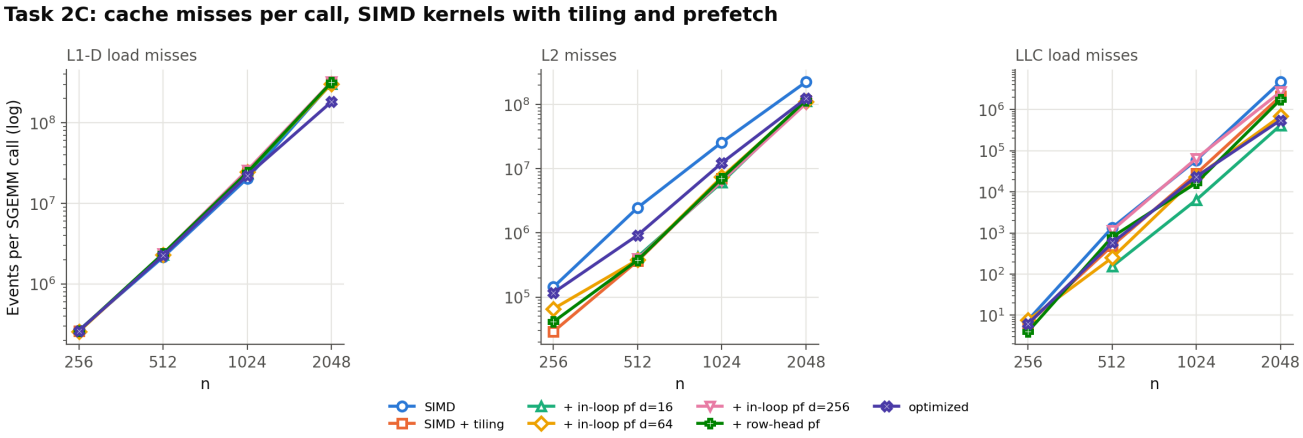


Figure 9: Cache misses per SGEMM call (log scale) for the SIMD kernels with tiling and the two prefetch styles.

Configuration at $n = 1024$ / 2048	Instr. (M)	Cycles (M)	L1-D miss (M)	L2 miss (M)	LLC miss (M)	Speedup
SIMD	259 / 2055	98.6 / 1206	20.4 / 310	25.8 / 225	0.06 / 4.69	39.5 / 24.0
SIMD + tiling	263 / 2072	89.9 / 1017	23.7 / 313	7.0 / 116	0.03 / 2.10	44.6 / 31.0
+ in-loop pf d=16, T0	331 / 2627	96.9 / 1051	23.6 / 305	6.1 / 113	0.01 / 0.42	41.7 / 31.5
+ in-loop pf d=64, T0	331 / 2631	95.0 / 1009	24.6 / 304	7.5 / 111	0.02 / 0.69	41.2 / 33.3
+ in-loop pf d=256, T0	331 / 2626	100.3 / 1070	25.6 / 321	6.5 / 104	0.07 / 2.69	39.8 / 28.9
+ in-loop pf d=64, T1	331 / 2628	107.4 / 1386	22.8 / 298	7.3 / 113	0.02 / 0.73	38.4 / 23.9
+ in-loop pf d=64, T2	332 / 2628	107.3 / 1392	22.8 / 298	7.2 / 115	0.02 / 0.73	38.6 / 23.8
+ in-loop pf d=64, NTA	331 / 2628	108.4 / 1274	24.7 / 309	19.7 / 450	0.02 / 0.96	37.4 / 25.4
+ row-head pf (2 lines), T0	268 / 2090	90.5 / 1020	24.1 / 317	7.1 / 116	0.02 / 1.75	44.0 / 30.9
optimized (B panel + K blocks)	260 / 2105	90.0 / 816	22.1 / 181	12.3 / 124	0.02 / 0.55	45.9 / 40.5

Tiling divides L2 misses by 3.7 at $n = 1024$ and halves LLC misses at $n = 2048$; that is the change that matters. In-loop prefetching reduces LLC misses further at $n = 2048$ (2.10 M to 0.42–0.69 M) and there it also lowers the cycle count slightly (1009 M against 1017 M at $d = 64$), but at $n = 1024$ it raises cycles by 6–12% with L2 misses already at the tiled level: it is removing misses that no longer cost anything while adding 26% more instructions (39 M prefetches plus their address arithmetic) to the load ports. As in Task 2A, T1/T2 and especially NTA multiply L2 misses (19.7 M and 450 M against 7.5 M and 111 M for T0) and cost 12–38% more cycles than T0. The optimized kernel's K-blocking shows in the L1 column at $n = 2048$: 181 M misses instead of 313 M.

Execution time and speedup versus prefetch distance and fill level

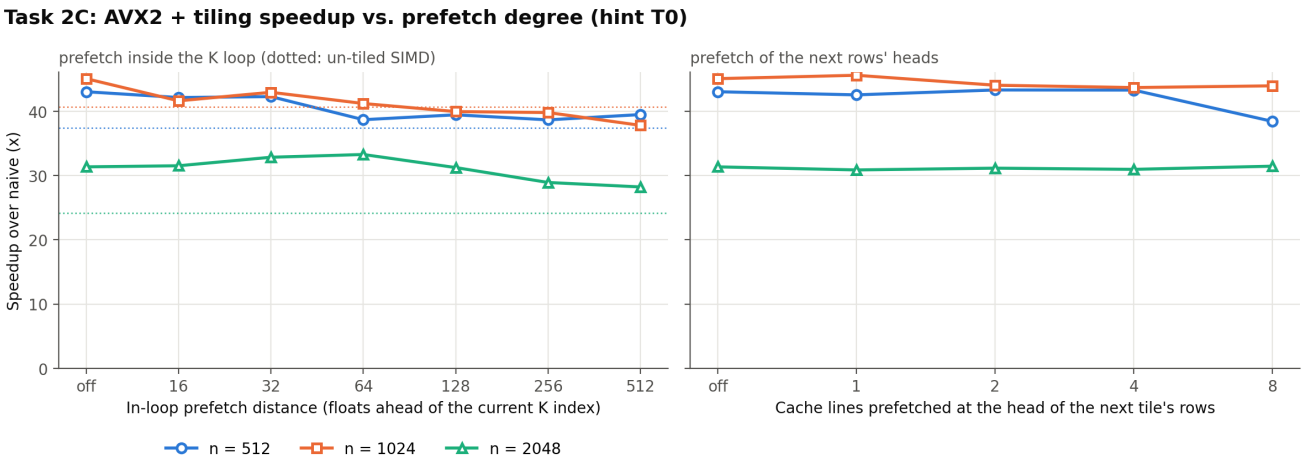


Figure 10: AVX2 + tiling speedup versus prefetch degree (README plot 2): in-loop distance (left) and number of row-head lines (right), hint T0. Dotted lines mark the un-tiled SIMD stage.

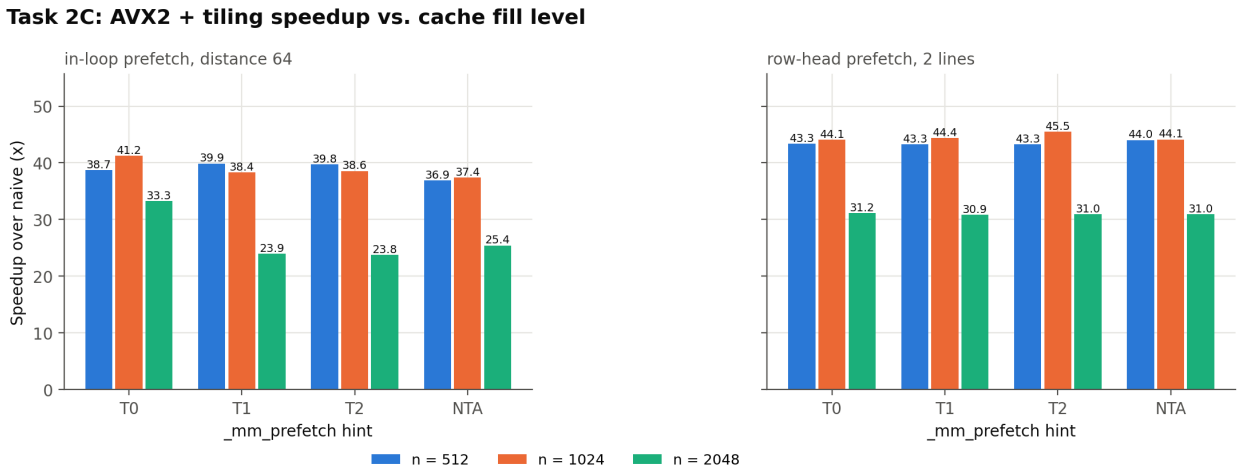


Figure 11: AVX2 + tiling speedup versus cache fill level (README plot 3), for in-loop prefetch at $d = 64$ (left) and row-head prefetch of two lines (right).

n	Configuration (AVX2 + tiling)	T0	T1	T2	NTA
1024	no prefetch	45.1×	–	–	–
1024	in-loop $d = 16$ / 64 / 256	41.7 / 41.2 / 39.8×	39.4 / 38.4 / 38.0×	39.0 / 38.6 / 37.7×	37.2 / 37.4 / 35.6×
1024	row-head $\ell = 1$ / 2 / 8	45.6 / 44.1 / 44.0×	44.4 / 44.4 / 44.2×	44.4 / 45.5 / 43.0×	47.5 / 44.1 / 41.5×
2048	no prefetch	31.4×	–	–	–
2048	in-loop $d = 16$ / 64 / 256	31.5 / 33.3 / 28.9×	27.9 / 23.9 / 21.9×	27.9 / 23.8 / 21.6×	26.9 / 25.4 / 22.7×
2048	row-head $\ell = 1$ / 2 / 8	31.0 / 31.3 / 31.6×	32.0 / 31.0 / 31.8×	31.4 / 31.1 / 31.5×	31.4 / 31.1 / 30.8×

Prefetching inside the vectorised K loop **lowers** the speedup at every distance for $n \le 1024$ (from $45.1\times$ to 38–$43\times$ with T0, 5–15%) and gains a little at $n = 2048$ ($33.3\times$ at $d = 64$ against $31.4\times$ without, $+6\%$), which is the one regime where the B panel no longer comes from the L3. The inner loop already issues seven loads per twelve FMAs; adding seven prefetch micro-operations per line competes with them for the load ports, and while the data is L3-resident there is nothing to gain because the hardware prefetcher streams within each row. Distances beyond 128 floats hurt at $n = 2048$ since seven streams prefetched half a kilobyte ahead push lines out of the 32 KiB L1, and T1, T2 and NTA lose 20–30% there because the prefetched line is fetched a second time. The sparse row-head prefetch behaves differently: one or two lines per upcoming row cost only 0.5 M prefetch instructions at $n = 1024$ ($+1.7\%$ instructions, $+0.7\%$ cycles) and keep the tiled kernel at 44–$47\times$; this is the form used by the submitted stages.

Optimal parameters

- **Prefetch distance:** for $n \le 1024$ the best in-loop distance is "none" ($45.1\times$ against $43.0\times$ for the best distance, $d = 32$); at $n = 2048$ an in-loop distance of 64 floats with T0 is best ($33.3\times$, $+6\%$). The prefetch that helps at every size is one to two lines at the head of the next tile's rows.
- **Cache fill level:** T0. For row-head prefetch all hints are equivalent; for in-loop prefetch T0 is best at every size and at $n = 2048$ it keeps $33.3\times$ while T1/T2/NTA fall to 23.8–$25.4\times$ because the line is fetched a second time by the demand load.

Effect of the hardware prefetchers

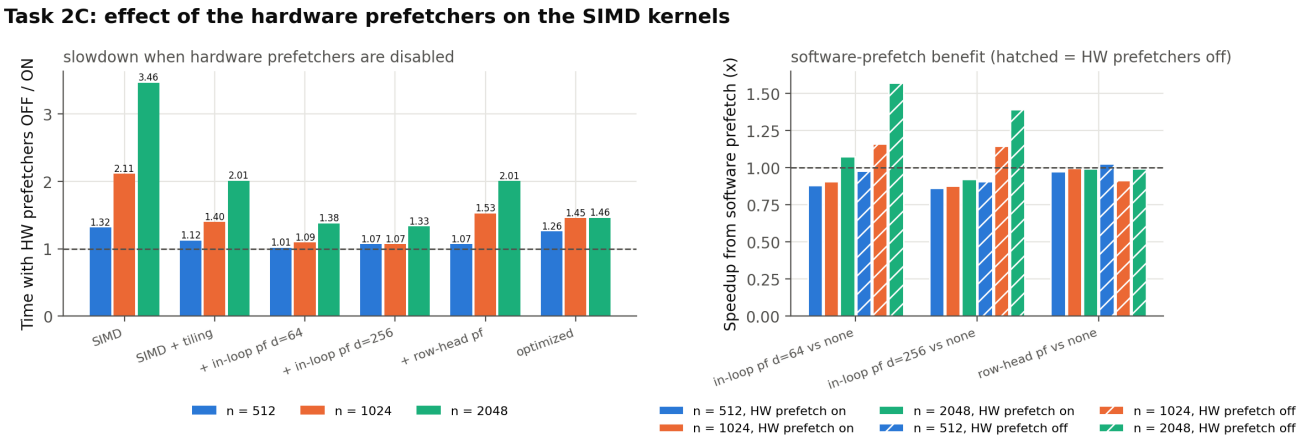


Figure 12: Left: slowdown of the SIMD configurations with all hardware prefetchers disabled. Right: speedup from each software-prefetch style relative to the tiled kernel without prefetch, prefetchers on (solid) and off (hatched).

Configuration	$n = 512$ (ms)		$n = 1024$ (ms)		$n = 2048$ (ms)	
	HW on	HW off	HW on	HW off	HW on	HW off
SIMD	9.0	11.9	69.4	146.6	973	3370
SIMD + tiling	7.5	8.5	62.3	87.1	750	1509
+ in-loop pf d=64	8.5	8.7	68.6	75.0	697	960
+ in-loop pf d=256	8.7	9.3	71.0	76.0	813	1083
+ row-head pf (2 lines)	7.7	8.2	62.5	95.3	755	1517
optimized	7.8	9.7	61.6	89.7	581	850

The vectorised kernels depend on the hardware prefetchers far more than the scalar one: without them the un-blocked SIMD kernel slows by $2.1\times$ at $n = 1024$ and $3.4\times$ at $n = 2048$, because it has the highest bandwidth demand per instruction. Tiling reduces the dependence to $2.0\times$ and the optimized kernel's K-blocking to $1.5\times$, since both replace DRAM streams by L1/L2 reuse. In that state the in-loop software prefetch finally helps: at $n = 2048$ it cuts the tiled kernel from 1509 ms to 960 ms ($1.57\times$ at $d = 64$; $1.39\times$ at $d = 256$, whose longer reach overflows the L1) and its LLC misses at $n = 1024$ from 0.40 M to 0.03 M per call, whereas the row-head prefetch, which covers only the first lines of each row, gives nothing (1517 ms). This confirms the interpretation of the previous subsection: the in-loop prefetch is redundant with the L2 streamer and L1 next-line prefetchers, and its cost is visible precisely because they are already doing the job.

2C.2 The Task 2B analyses with prefetching

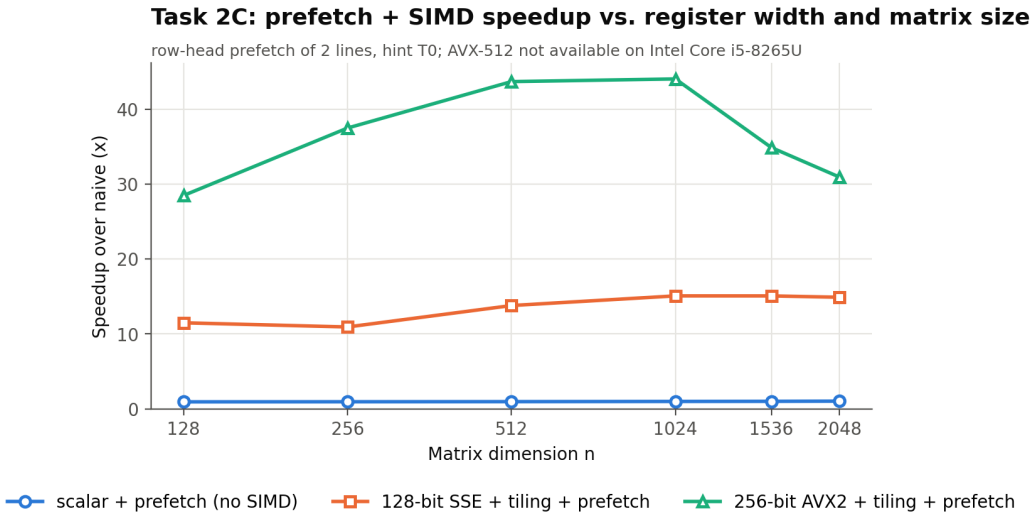


Figure 13: Speedup over the scalar kernel for the 128-bit and 256-bit tiles with tiling and row-head prefetch (2 lines, T0), and for the scalar kernel with prefetching alone.

Metric (prefetch on)	Width	Matrix size (M, N, K) $(256, 256, 256)$	$(512, 512, 512)$	$(1024, 1024, 1024)$	$(2048, 2048, 2048)$
No SIMD (scalar + prefetch): instructions (M)	–	154.2	1230.9	9834.6	78592.0
No SIMD (scalar + prefetch): time (ms)	–	42.48	347.06	2831.21	22718.13
SIMD + tiling + prefetch: instructions (M)	128-bit	16.3	125.6	986.1	7829.2
SIMD + tiling + prefetch: time (ms)	128-bit	3.72	24.44	187.54	1579.21
Speedup over naive	128-bit	10.9×	13.8×	15.1×	14.9×
SIMD + tiling + prefetch: instructions (M)	256-bit	5.1	35.8	267.7	2089.6
SIMD + tiling + prefetch: time (ms)	256-bit	1.08	7.72	64.24	760.48
Speedup over naive	256-bit	37.5×	43.7×	44.0×	30.9×

With tiling and prefetch the ordering of Task 2B is unchanged: 256-bit stays 2.1–$3.4\times$ ahead of 128-bit. Tiling helps the 256-bit kernel most ($24\times$ to $31\times$ at $n = 2048$) and the 128-bit kernel least ($13.6\times$ to $14.9\times$), because only the faster kernel was memory-bound. The instruction counts barely change relative to Table 2.1 of Task 2B (+1–3% from the tail handling of the panels and the few row-head prefetches), so the whole gain comes from cache behaviour, not from instruction reduction.

Questions (both techniques applied)

2A-Q1. What trend do you observe in speedup with different matrix sizes?

With SIMD present, speedup over naive rises with size up to $n = 1024$ (37–$46\times$) as fixed costs amortise, then the un-blocked kernel collapses beyond the L3 ($24\times$ at 2048) while the tiled kernels hold $31\times$ and the K-blocked optimized kernel $40\times$. The size trend is set by cache blocking; software prefetching itself moves the result by a few percent either way when the hardware prefetchers are on.

2A-Q2. What is the best prefetch distance?

Inside the vectorised loop, none for $n \leq 1024$, where every in-loop distance costs 5–15%; at $n = 2048$, $d = 64$ floats with T0 gains 6% over no prefetching. The prefetch that pays at every size is one to two cache lines at the head of the next tile's rows, which is what `matmul_prefetch` and `matmul_optimized` do. With the hardware prefetchers disabled the in-loop distance of 64 floats is best ($1.57\times$).

2A-Q3. At what cache fill level do you achieve the maximum speedup?

T0. For the row-head prefetch every hint gives the same 44–$47\times$; for the in-loop prefetch at $n = 2048$ only T0 avoids a second fetch of each line ($33.3\times$ against 23.8–$25.4\times$ for T1, T2 and NTA), and NTA multiplies L2 misses by about 3.

2B-Q1. What trends do you observe in speedup for different combinations of matrix sizes and SIMD widths?

The same as without prefetching, with the large-size cliff removed for the wider kernel: 128-bit stays flat at 11–$15\times$, 256-bit reaches $44\times$ at $n = 1024$ and, thanks to tiling, keeps $31\times$ at $n = 2048$ instead of $24\times$. Prefetching changes these numbers by at most a few percent.

2B-Q2. For which SIMD width do you achieve the maximum speedup?

256-bit AVX2 at every size ($44.0\times$ at $n = 1024$, $30.9\times$ at $n = 2048$ with tiling and prefetch; $45.9\times$ and $40.5\times$ with the optimized kernel's additional K-blocking).

Final performance summary

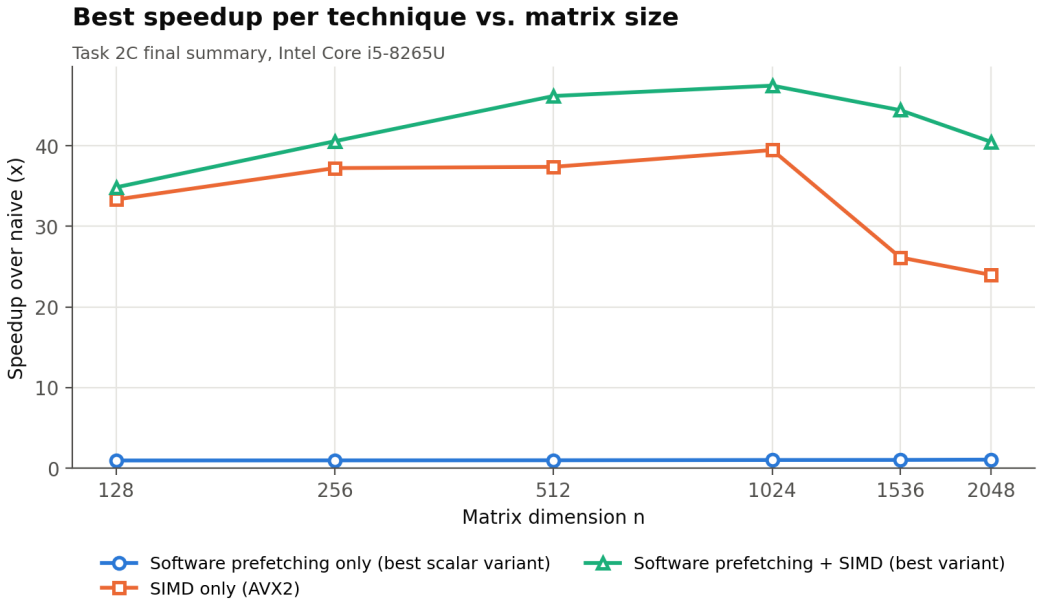


Figure 14: Best speedup achieved by each optimization technique versus matrix size.

Technique (best variant)	n=128	256	512	1024	1536	2048
Software prefetching only (scalar)	0.94×	0.96×	0.97×	1.01×	1.01×	1.05×
SIMD only (AVX2)	33.3×	37.2×	37.4×	39.5×	26.1×	24.0×
Software prefetching + SIMD (+ tiling)	34.8×	40.6×	46.2×	47.5×	44.4×	40.5×

README deliverables

The README asks for four plots with a justification of each technique's gain or loss:

1. **Speedup vs. matrix size** (`plots/task2_speedup_vs_size.png`, Section 2C.1): SIMD alone gains 33–$40\times$ but loses to memory bandwidth beyond the L3; tiling recovers $31\times$ at $n = 2048$ and K-blocking $40\times$.
2. **Speedup vs. prefetch degree** (`plots/task2_prefetch_degree.png`, Section 2C.1): in-loop prefetching loses 5–15% at $n \leq 1024$ because it competes with the FMA stream for load ports while the hardware prefetcher already covers the rows, and gains 6% only at $n = 2048$; row-head prefetching of one or two lines is neutral to slightly positive at every size.

3. **Speedup vs. prefetch level** (`plots/task2_prefetch_level.png`, Section 2C.1): T0 is best or equal everywhere; T1/T2/NTA lose because the prefetched line has to be fetched twice.
4. **Speedup vs. SIMD width** (`plots/task2_simd_width.png`, Section 2B.2): 256-bit beats 128-bit by $2.6\times$ in cache and $1.8\times$ out of cache; 512-bit is unavailable on this processor.

Conclusions

1. The scalar SGEMM loop is latency-bound, not memory-bound: software prefetching adds 82% instructions and under 1% cycles, and only at $n = 2048$ gains 5%. Its best parameters are a distance of 256 floats and hint T0, and NTA measurably increases L2 traffic.
2. Disabling the hardware prefetchers doubles the naive kernel's time at $n = 2048$, and software prefetching then recovers all of it ($2.0\times$): the two mechanisms are substitutes for these unit-stride streams.
3. Explicit AVX2 with a 4×3 register tile is the decisive optimization: $20.8\times$ fewer instructions and 37–$40\times$ speedup up to $n = 1024$. It turns the kernel memory-bound, so speedup falls to $24\times$ at $n = 2048$.
4. In the vectorised kernel, cache tiling (L2 misses $\div 3.7$) and K-blocking (L1 misses $\div 1.7$ at $n = 2048$) are what restore performance; in-loop software prefetching costs 5–15% while the data is L3-resident because the hardware prefetchers already stream each row, helps 6% only once the operands exceed the L3, and a sparse prefetch at row boundaries is the form that pays at every size.
5. The submitted `matmul_optimized` reaches $43.6\times$ in the graded harness, $45.9\times$ at 1024^3 and $40.5\times$ at 2048^3 in the controlled sweeps, computes correct results for arbitrary shapes and strides, and reproduces `llama.cpp`'s output exactly.

The submitted code modifies only the three permitted source files and preserves the supplied build system and flags. The experiment variants are compile-time options of those same files that are inactive in the graded build; the scripts, binaries and raw results that produced this report live in `experiments/`.